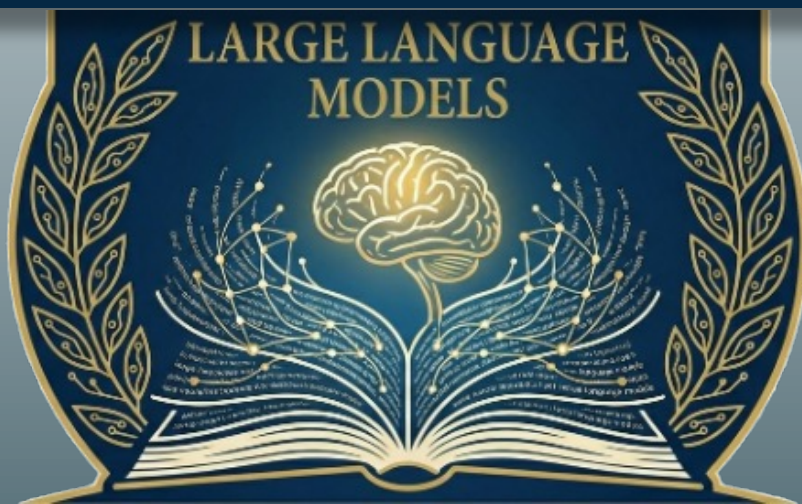


Quantization

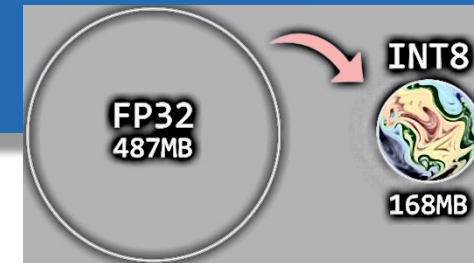


Mohammad Javad Dousti

Source:

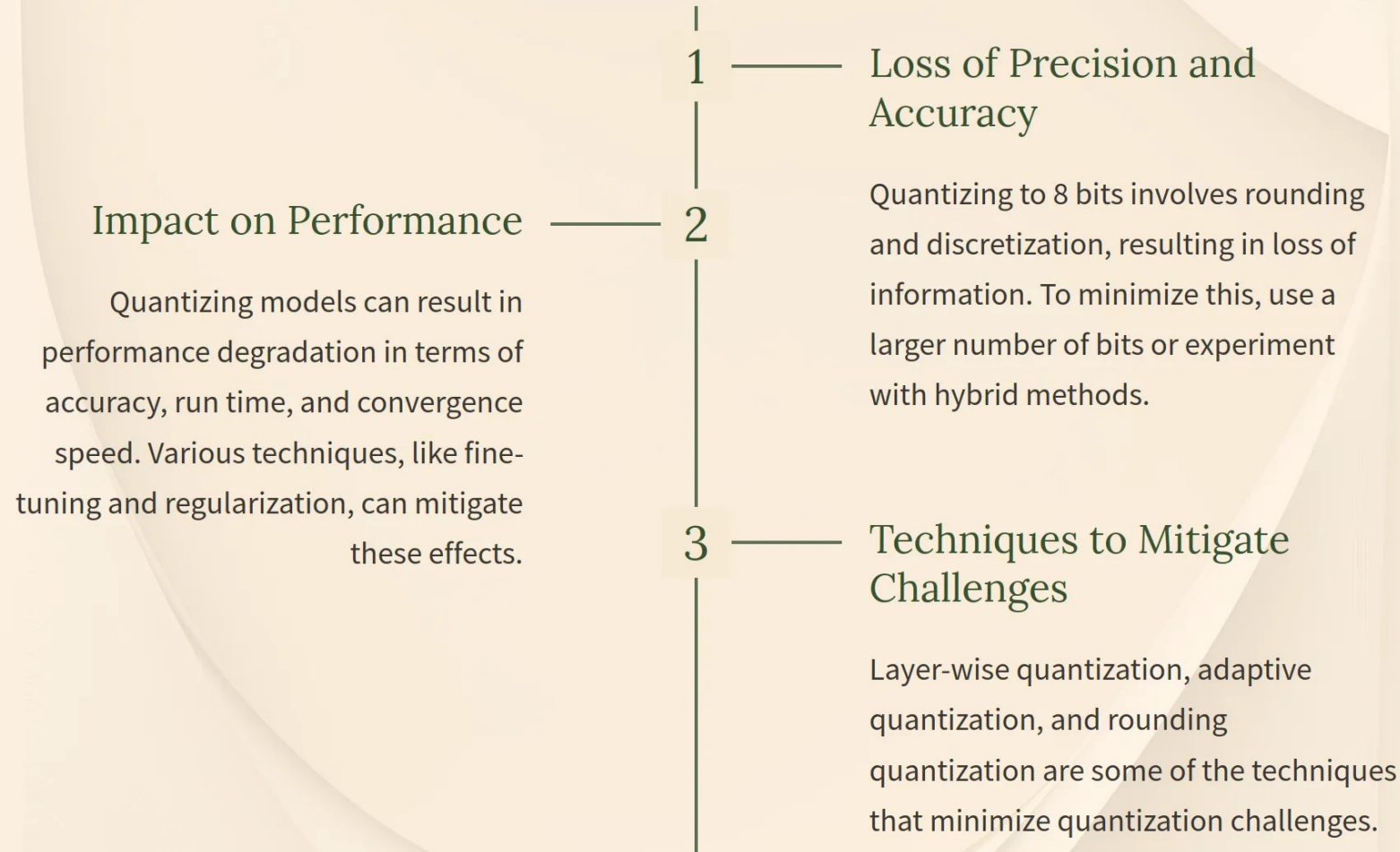
* LLM course (<https://github.com/mlabonne/llm-course>) by Maxime Labonne.

Introduction



- ❑ LLMs need extensive computational requirements.
- ❑ To save memory, weights can be stored using *lower-precision* data types through a process known as **quantization**.
- ❑ Quantization types:
 - **Post-Training Quantization (PTQ)**: Weights of a trained model are converted to lower precision without necessitating any retraining. Although easy to implement, PTQ is associated with potential performance degradation.
 - **Quantization-Aware Training (QAT)**: Incorporates the weight conversion process during the pre-training or fine-tuning stage, resulting in enhanced model performance. It is computationally expensive and demands representative training data.

Challenges of Model Quantization



Floating Point Representation: Quick Reminder

32-bit float (FP32)



$$(-1)^0 \times 2^{128-127} \times 1.5707964 = 3.1415927$$

16-bit float (FP16)



$$(-1)^0 \times 2^{16-15} \times 1.571 = 3.141$$

bf16 (BF16)



$$(-1)^0 \times 2^{128-127} \times 1.5703125 = 3.140625$$

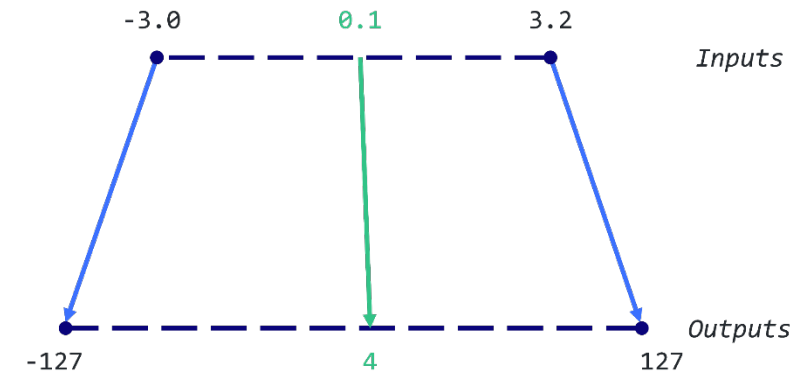
Naïve 8-bit Quantization: **absmax** quantization

- ❑ **Goal:** Quantize FP32 tensor $\mathbf{X}$ to an INT8 tensor $\mathbf{X}_{\text{quant}}$
- ❑ **Quantization:** The original number is divided by the absolute maximum value of the tensor and multiplied by a scaling factor (127) to map inputs into the range $[-127, 127]$.

$$X_{\text{quant}} = \text{round} \left(\frac{127}{\max|X|} \times X \right)$$

- ❑ **Dequantization:** To retrieve the original FP32 values, the INT8 number is divided by the quantization factor, acknowledging some loss of precision due to rounding.

$$X_{\text{dequant}} = \frac{\max|X|}{127} \times X_{\text{quant}}$$



Naïve 8-bit Quantization: zero-point quantization

- ❑ Assume the input distribution is asymmetric (e.g., ReLU only gives positive output)
- ❑ Through an affine transformation, quantization is done.

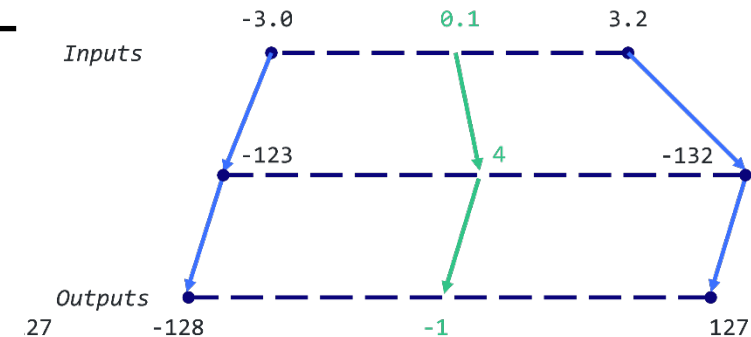
$$scale = \frac{255}{\max(X) - \min(X)}$$

$$zeropoint = -\text{round}(scale \times \min(X)) - 128$$

- ❑ Quantization:

$$X_{quant} = \text{round}(scale \times X + zeropoint)$$

$$X_{dequant} = \frac{X_{quant} - zeropoint}{scale}$$



8-bit Quantization with LLM.int8()

- ❑ Outlier issue
- ❑ LLM.int8() is a solution to the outlier problem.
 - Relies on a vector-wise (**absmax**) quantization scheme
 - Introduces mixed-precision quantization.
 - Outlier features are processed in a FP16 format to retain their precision, while the other values are processed in an INT8 format.
 - As outliers represent about **0.1%** of values, this effectively reduces the memory footprint of the LLM by almost 2x.

How LLM.int8() works

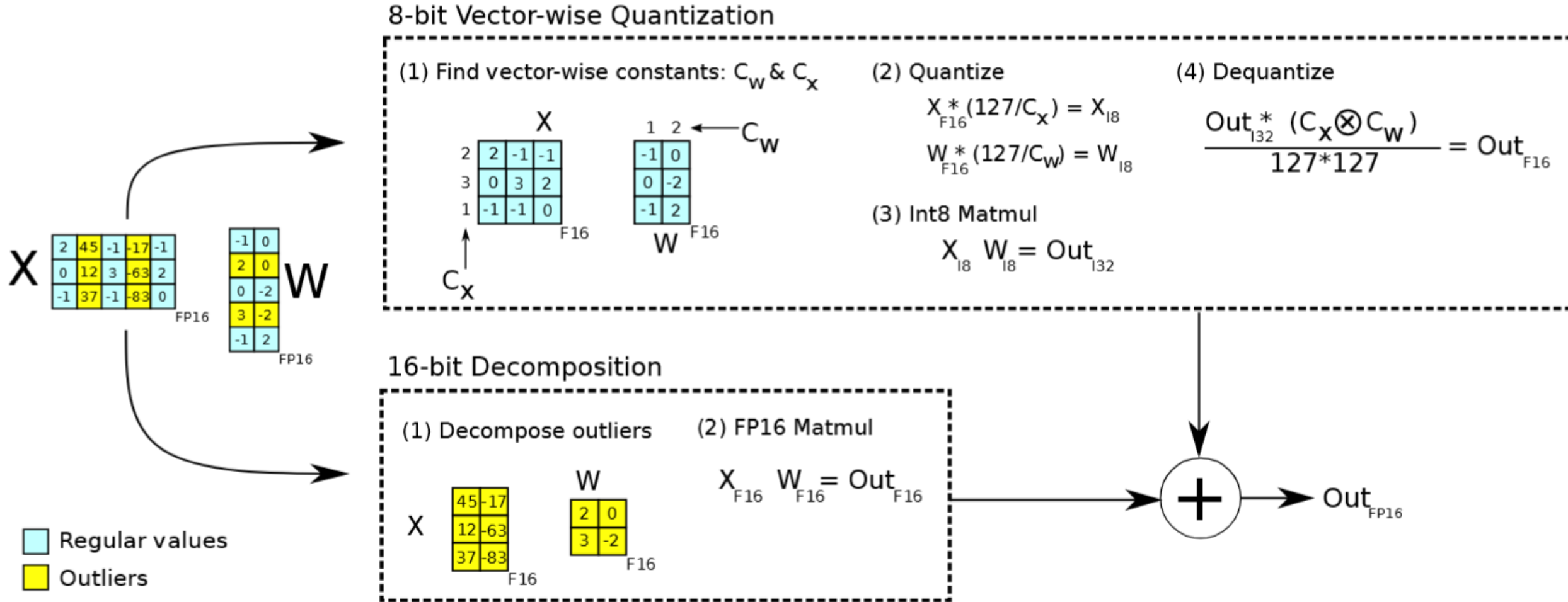
1. Extract columns from the input hidden states X containing outlier features using a custom threshold.
2. Perform the matrix multiplication of the outliers using FP16 and the non-outliers using INT8 with vector-wise quantization (row-wise for the hidden state X and column-wise for the weight matrix W).
3. Dequantize the non-outlier results (INT8 to FP16) and add them to the outlier results to get the full result in FP16.

$$\mathbf{X}_{F16} = \begin{bmatrix} 2 & 45 & -1 & 17 & -1 \\ 0 & 12 & 3 & -63 & 2 \\ 1 & 37 & 1 & -83 & 0 \end{bmatrix} \cdot \mathbf{W}_{F16} = \begin{bmatrix} -1 & 0 \\ 2 & 0 \\ 0 & -2 \\ 1 & -2 \\ -1 & 2 \end{bmatrix}$$

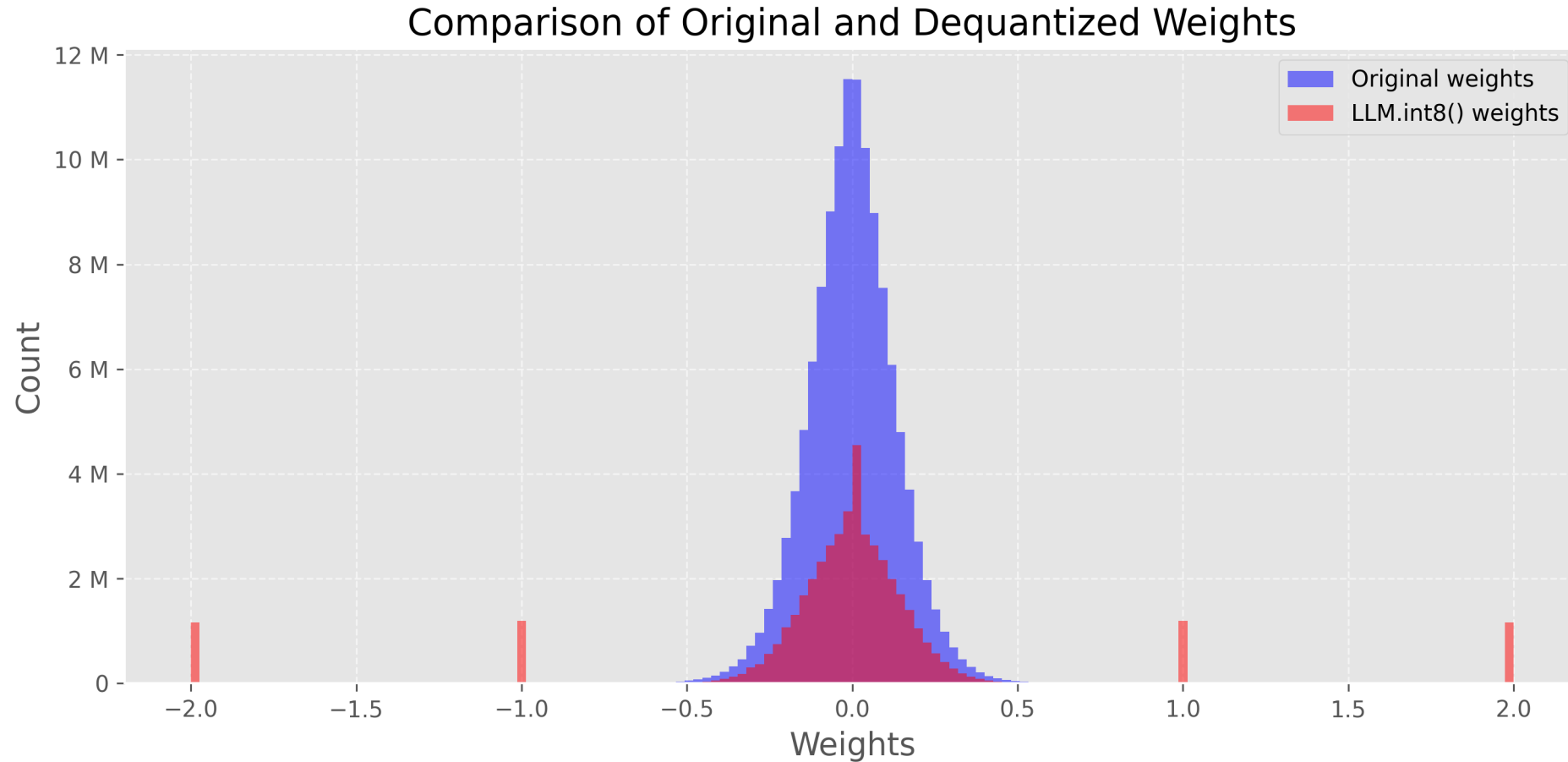
Outliers

Corresponding rows

LLM.int8() (cont'd)



LLM.int8() (cont'd)

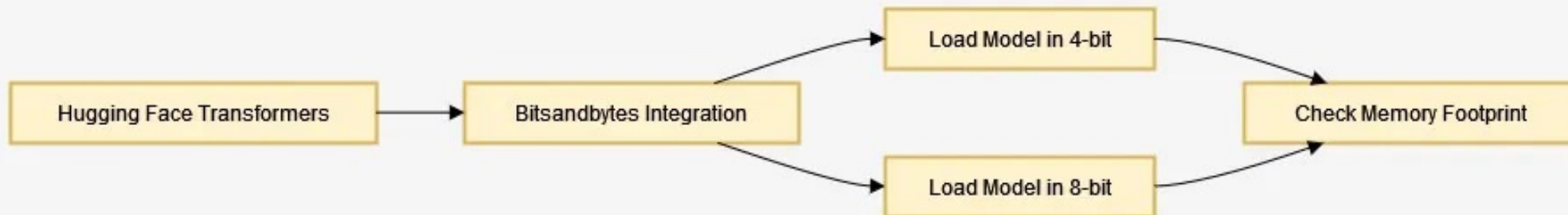


Spikes around -2, -1, 0, 1, 2, etc. correspond to the parameters stored in the INT8 format (non-outliers).

Implementation through BitsAndBytes Library

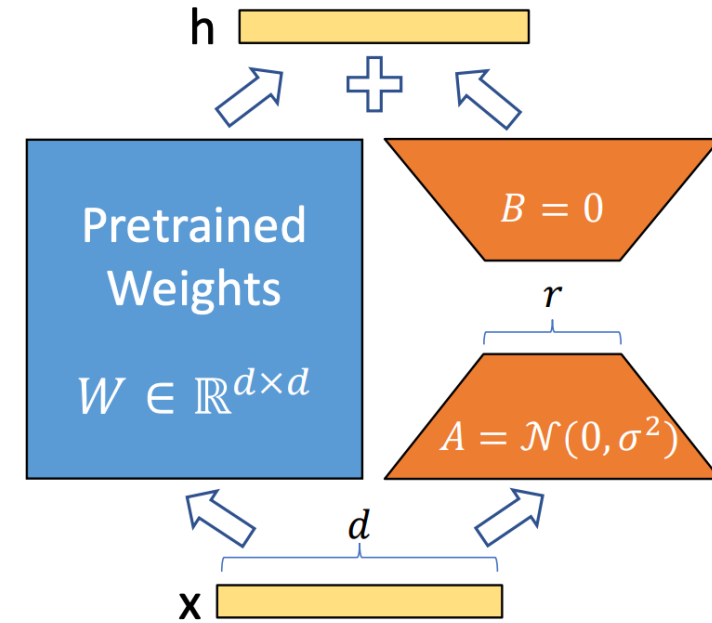
```
quantization_config = BitsAndBytesConfig(load_in_8bit=True)

model_int8 = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    quantization_config=quantization_config,
)
print(f"Model size: {model_int8.get_memory_footprint():,} bytes")
```



Low Rank Adaptation (LoRA) Reminder

- Linear layers: $h = Wx$
- With LoRA: $h = Wx + \Delta W x = W_0x + sBAx$
- Hyperparameters:
 - rank r
 - List of layers to apply to
 - Scaling factor s
- At inference time, merge LoRA weights:
 $W = W_0 + BA$. Same inference cost!



LoRA (Hu *et al*, 2021)

QLoRA

- ❑ PEFT with LoRA (assume 0.4% more params):
 - Base weights: 2 bytes * 13B = 26GB
 - LoRA weights: 26GB * 0.4% = 0.10GB
 - Gradients: 52GB * 0.4% = 0.20 GB
 - Optimizer State = 12 bytes/ param * 13B * 0.4% = 0.62GB
 - Total = 26.92 GB + Activation memory
- ❑ QLoRA (4bit):
 - Base weights: 0.5 bytes * 13B = 6.5GB
 - LoRA weights: 26GB * 0.4% = 0.10GB
 - Gradients: 52GB * 0.4% = 0.20 GB
 - Optimizer State = 12 bytes/ param * 13B * 0.4% = 0.62GB
 - Total = 7.42 GB + Activation memory

QLoRA: TradeOffs

- ❑ Throughput?
 - Quantization has worse throughput. (~30% for single-GPU)
 - Why? Extra dequantization step in forward pass! (T_{math})
- ❑ Performance?
 - Dettmers et al. show that QLoRA can match full-parameter finetuning
- ❑ QLoRA – to be used in a consumer hardware setup with 1-2 GPUs
 - Ex. 2 RTX 4090s (Total: 48GB HBM) to finetune a 33B model
- ❑ Quantization almost never needed with a multi-GPU (4+ GPUs) setting.

Optimization	Memory	Throughput	Model Performance
Quantization	↓	↓	●